

TECHNICAL DESIGN DOCUMENT

Evidence, by design.

RAG Workbench · End-to-end architecture

INGEST → RETRIEVE → VERIFY → OBSERVE

Grounded answers. Inspectable decisions. Bounded execution.

A browser workbench for testing enterprise document Q&A, tracing every released claim to evidence, and diagnosing retrieval, model and verification failures.

01 / SYSTEM DESIGN**From file to cited answer**

Boundaries, data contracts, endpoints and execution sequences. Pages 02-06.

02 / TECH STACK**Technology with a purpose**

Frameworks, model adapters, storage, telemetry and evaluation. Pages 07-08.

03 / PRODUCT FEATURES**Delivered now; next in line**

MVP capabilities, user journeys and Phase 2 acceptance criteria. Pages 09-10.

04 / SCALE & SECURITY**Production evolution**

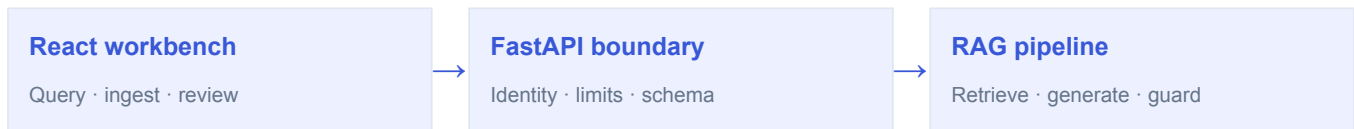
Horizontal scaling, isolation, failover, limits and rollout gates. Pages 11-14.

Baseline: inspected source in `/Users/sandeepkumarboda/projects/enterprise-rag`, 30 September 2026. Git HEAD 339862a plus working-tree updates. Defaults in this document describe code configuration, not private environment overrides.

01 / SYSTEM DESIGN

A modular monolith, with clear seams

Keep the query decision path cohesive; separate infrastructure and isolate risky file parsing.



Inside the API process

Dependency-injected pipeline, model gateway, local inference adapters, CPU worker pool, bounded answer cache and SQLite repository.

External model endpoint

HTTPS Responses-compatible API. Server-owned credentials and endpoint routes. Only selected evidence and the query cross this boundary.

Local durable state

SQLite WAL: chunks, float32 vectors, lexical tokens, corpus revisions and review decisions. No separate vector database today.

Operational services

OTel Collector → Jaeger traces; spanmetrics → Prometheus → Jaeger Monitor. These are observation services, not business microservices.

Why this deployment shape?

The target use case is controlled enterprise document Q&A: an operator ingests documents, tests questions, inspects citations and escalates ambiguous results. A single backend keeps identity, retrieval snapshots and citation checks in one execution boundary. Atomic document replacement avoids synchronizing two independent indexes during MVP delivery.

Isolation where it matters

File parsing runs in a disposable subprocess with a bounded lifetime and reduced environment. Neural work runs off the async event loop. Neither mechanism is a separately deployable microservice.

Trade-off and exit point

Exact search and in-process model locks favor debuggability over throughput. Split storage and inference only after measured CPU queues, corpus growth or availability requirements justify the operational cost.

Design invariant

No unverified draft is released. Empty evidence, invalid citations, unresolved conflicts and provider failures remain explicit outcomes; a faster answer is not allowed to bypass the guard.

01 / SYSTEM DESIGN

Data that preserves provenance

The authoritative source is the extracted text and its immutable chunk version, not model-generated wording.

Entity / persistence	Key fields and relationships
Principal · request scoped	Server-owned tenant, groups, roles and hashed subject. Bearer token maps to this identity; the client cannot choose a tenant.
Document · ingestion contract	id, title, source, text, groups. A logical document is represented by its chunks; there is no separate documents table or retained original-file store.
Chunk · chunks	Composite primary key (tenant, id); document_id; JSON payload with version, text, offsets, source and groups; vector BLOB; token JSON. Index on (tenant, document_id).
Corpus revision · corpus_versions	Tenant primary key + integer revision. Incremented in the same transaction as ingest/delete. Used for answer-cache invalidation.
Index identity · metadata	Schema/encoder identity fingerprint. Incompatible embedding identity fails closed; rebuild into a separate database.
Review · reviews	request_id, tenant, originating groups, payload, state, decision, created_at. Payload includes question, authorized contexts and only a validated draft if one exists.
Answer · response contract	status, answer_style, claims, citations, contexts, reasons, faithfulness, usage, stage timings, trace_id and request_id. Not every answer is durably stored.

Evidence relationships



A hit adds fusion and rerank scores to a chunk. A claim carries 1-4 evidence references; a generation contains at most 8 claims. Citations resolve to stored source spans. For uploaded files, offsets refer to the extracted text representation, not PDF pixel coordinates or original byte offsets.

Consistency and cache boundary

Prepare chunks and embeddings before BEGIN IMMEDIATE. Replace the document's previous chunks and increment the revision atomically; any write failure rolls back. Cache scope includes identity, query, policy/configuration and corpus revision. Only released answers enter the bounded cache.

01 / SYSTEM DESIGN

API contracts and trust boundaries

The browser uses a configurable API client; authorization and tuning policy remain on the server.

Method and route	Contract / access
POST /query	JSON {text}; 3-2,000 characters. Requires query role. Returns the structured Answer contract, including answered / abstained / review.
POST /ingest	JSON {documents:[...]}; 1-20 documents. Requires ingest role. Returns indexed_chunks. Batch replacement is one transaction.
POST /ingest/file	Multipart: one file; optional document_id, title, groups as JSON string array. Requires ingest role. Returns document_id, source, indexed_chunks, extracted_characters and warnings.
GET /ingest/formats	Public capability discovery: extensions, file/character/page/sheet limits and parse timeout. UI validation mirrors server capabilities.
DELETE /documents/{document_id}	Requires ingest role. Removes the logical document within the caller's tenant; returns deleted_chunks and bumps revision when changed.
GET /reviews	Requires review role. Loads up to 100 tenant records then applies originating-group visibility. Not a paginated queue yet.
POST /reviews/{request_id} /decision	Requires review role. {decision: approve reject, note}. Pending-only transition; returns {recorded:true}. Does not automatically release an answer.
GET /health/live / /health/ready	Public liveness/readiness. Readiness probes local DB access and reports mode, pipeline revision and answer style; it does not test provider availability.

Transport semantics

401: invalid identity; 403: role/group denial; 413: oversized request; 415: unsupported upload; 422: schema/parsing failure; 429: rate limit; 503: busy or unavailable. An abstention/review can be a successful HTTP 200 response.

Frontend contract

Fetch for JSON, XHR for real upload byte progress, Zod response validation, cancellation and safe messages. VITE_API_BASE_URL defaults to /api; Vite proxies local requests. Top-K and temperature are not accepted query fields.

Correlation

Headers expose x-request-id and x-trace-id. The pipeline's request ID is distinct from the ingress request ID; trace_id joins HTTP, pipeline, stages and provider attempts. Treat IDs as correlators, not credentials.

01 / SYSTEM DESIGN

Ingestion: bytes to searchable evidence

Validate early, isolate extraction, then publish the lexical and vector representations together.

01 Browser queue

Drag/drop or select files. Preview filename and size, set groups and an optional stable ID. Client checks limits; each file is submitted independently. 100% upload progress means server processing, not successful indexing.

02 Ingress and metadata validation

Bound request bytes and concurrency; authenticate and require ingest role. Reject unexpected multipart fields, unsupported extensions and groups outside the caller's identity. Sanitize the filename; default ID/source are filename hashes.

03 Private temporary file → extraction subprocess

Write a mode-0600 temporary file. Run rag.file_parser without model credentials. Enforce a 30-second default parser deadline, kill/reap on timeout and clean up temporary data.

04 Document → chunks → embeddings

Validate extracted content. Sentence, paragraph or semantic splitting preserves text offsets; default target is 700 characters with 80-character overlap. Encoder token limits also constrain chunks. Compute normalized vectors and lexical tokens.

05 Atomic commit → UI result

Validate finite vectors and capacity before publishing. Replace previous chunks for the same tenant/document ID in one SQLite transaction; bump corpus revision. Return indexed count and parser warnings. A failed file does not roll back other successful file uploads.

Format	Extraction and deliberate limits
PDF	pdfplumber text + best-effort tables with page/row labels. No OCR. Scanned pages warn; entirely unreadable files fail. Encrypted files require decryption first.
DOCX / XLSX	Defused XML body paragraphs/tables; openpyxl sheet rows including hidden sheets. No macros, formula evaluation or link execution. Missing cached formula results are reported.
CSV / TXT / MD	Bounded decoding and row-labelled CSV parsing. Plain text/Markdown remains text; no HTML or script execution.

Default ceilings

10 MiB/file; 200,000 extracted characters; 100 PDF pages; 50 sheets; 10,000 rows; 200 columns; 100,000 cells; 40 MiB Office archive expansion. Reusing a filename-derived ID replaces that document; use custom IDs for explicit versions.

01 / SYSTEM DESIGN

Query: retrieval is only the beginning

The default live path selects source sentences, constructs claims server-side and then verifies them.



Execution sequence

- 1. Retrieve an authorized snapshot.** Tenant filter in SQL; group intersection before ranking. Embed the original question. Normalized vector dot products provide exact cosine similarity; custom Okapi BM25 supplies lexical matches. Fuse positive matches using reciprocal rank fusion, $1/(60 + \text{rank})$, into at most 30 candidates.
- 2. Rank and select dynamically.** Cross-encoder scores question against title + chunk. Keep scores $\geq \max(0.4, \text{best score} \times 0.6)$, up to 6 chunks. Remove heavy overlaps and retain whole passages within a conservative UTF-8 byte-based token bound. Scores are not calibrated probabilities.
- 3. Rewrite only when needed.** Default fallback rewriting runs only if context selection is empty. Up to two validated paraphrases preserve the original query, followed by another retrieval pass. Always/off modes are configurable.
- 4. Generate a strict contract.** Extractive mode asks the provider for source-span IDs and abstain. The server maps IDs back to exact source wording. Synthesis mode can generate paraphrases with references; both paths require schema validation.
- 5. Verify before release.** Check quote existence, citation offsets, safety, NLI support and contradictions. Unresolved conflicts can trigger bounded semantic adjudication. Synthesis can attempt one bounded repair; extractive mode avoids that extra generative repair loop.
- 6. Finalize deterministically.** Release cited claims only when accepted and no review policy applies. Persist review records when required. Return contexts, reasons, usage and timings even when the answer is withheld.

Answered

Grounding checks passed; render claim badges and source-linked citations. Cache reuse gets fresh request/trace metadata and no new provider charge.

Abstained / review

No context, model abstention or exhausted provider budget can abstain. Unsupported claims, invalid citations, unresolved conflicts, unsafe output or degraded ranking can require review.

02 / TECH STACK

A small stack with explicit responsibilities

Versions below are declared project ranges or pinned infrastructure images, not claims about the newest releases.

Layer	Technology	Why this choice / trade-off
Web application	React 19 · TypeScript 5.9 · Vite 6	Component-based inspection UI with typed contracts and fast local iteration. A client-rendered internal tool does not require SSR.
Presentation	Tailwind CSS 4 · Lucide	Responsive layout, light/dark theme and consistent icons. Styling remains separate from domain and API state.
Client validation	Zod 4 · Fetch · XHR	Validate untrusted response shapes at runtime; abort/time out requests. XHR exposes actual upload bytes; no fabricated progress.
HTTP service	Python 3.12-3.14 · FastAPI · Uvicorn	Async orchestration, dependency injection and explicit route contracts. CPU-heavy inference must leave the event loop.
Configuration	Pydantic 2 · pydantic-settings	Strict schemas and validated budget relationships; SecretStr protects accidental representation of configured secrets.
HTTP model gateway	HTTPX · Responses-compatible JSON	Reusable async client, strict output schemas and provider route abstraction. Compatibility must be checked for each endpoint/model.
Local ML	sentence-transformers 5 · PyTorch · NumPy	Pinned embedding, reranker and NLI adapters; normalized vectors and batch inference. CPU is default; acceleration needs benchmarking.
Durability	SQLite · WAL · FULL synchronous	Atomic replacement of vectors and lexical tokens plus review persistence. Simple local operations, but single-host write coordination and exact scan costs limit scale.
Parsing	pdfplumber · openpyxl · defusedxml · csv	Bounded format-specific extraction without Office execution. Table layout and scanned pages are not fully solved.
Delivery & checks	uv.lock · Docker Compose · pytest · Ruff · Vitest	Reproducible dependencies, regression coverage and frontend build/type checking. CI also runs offline evaluation.

DB decision

SQLite stores float32 vector blobs and token arrays; NumPy and custom BM25 score an authorized snapshot at query time. There is no ANN index, Elasticsearch, Redis, LangChain or managed vector store in the delivered data path. This makes the baseline easy to audit, not horizontally scalable by default.

02 / TECH STACK

Models, telemetry and quality gates

Observe the entire decision chain, and keep outcome quality distinct from HTTP availability.

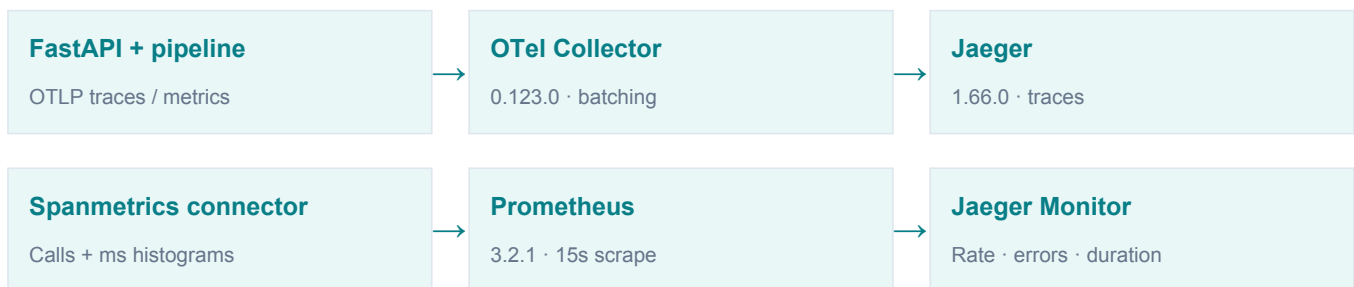
Embedding + ranking

all-MiniLM-L6-v2 supplies semantic embeddings; ms-marco-MiniLM-L6-v2 reranks pairs. Pinned model revisions, trust_remote_code=False and warmup support reproducibility.

Verification + generation

nli-deberta-v3-small estimates entailment/contradiction. The LLM model, price schedule and HTTPS endpoint are configurable. A provider name is not hard-coded as an architectural dependency.

Operational signal flow



The collector flushes span metrics every 5 seconds. Jaeger queries the normalized `traces_span_metrics` namespace. Charts need fresh traffic and multiple scrapes; old traces do not backfill metrics. Model token usage, known cost, retries, pipeline latency and outcome counters are separate application instruments. Monitor's service latency includes health traffic and is not automatically query-only latency.

Custom evaluation pipeline

Metric	Interpretation / gate
Faithfulness	Guard-derived support of released claims; null for unanswered cases. Gate: ≥ 0.95 aggregate, with separate success gating.
Answer relevance	Live model judge; demo uses lexical F1. Scores are not interchangeable. Gate: ≥ 0.70 .
Context precision / recall	Distinct retrieved document IDs versus labeled relevant IDs; not token-level recall or Ragas. Gates: ≥ 0.70 / ≥ 0.90 .
Operational acceptance	Success rate 1.0, nonempty/scored set, no per-case errors or uncertain billing; configured p95 latency and average known generation-cost limits.

Evidence, not an SLA

Recorded regression: 117 backend tests, 14 frontend tests and a passing build. Eight live source-selection samples met expected outcomes (median 3.07 s, range 1.46-3.68 s). This small sample does not establish production percentiles. Monitoring APIs were independently validated using health traffic. [S6]

03 / PRODUCT FEATURES

MVP: a complete inspection loop

Delivered features focus on making answers and failures inspectable, from ingestion through review.

01 · Query playground

Single-question submissions, loading/error states, session run history, sample prompts and live/demo visibility. Settings displayed in the UI do not create unsupported per-query tuning fields.

02 · Evidence-first answers

Claim badges, numbered citations and source quotes. Answer style distinguishes exact extracted wording from synthesis. Abstention and review remain visible rather than being disguised as successful answers.

03 · Run inspector

Retrieved chunks, fusion/rerank scores, stage latency, faithfulness, token/cost accounting, provider attempts, cache indicator and trace/request IDs.

04 · Document ingestion

Single-text and JSON batch inputs plus PDF/MD/XLSX/DOCX/CSV/TXT files. Drag/drop queue, size preview, progress, retry, cancellation, stable IDs, group assignment and extraction warnings.

05 · Human decision workflow

Authorized reviewers inspect flagged records and record approve/reject with a note. Decisions are durable and pending-only; approval does not automatically publish a response.

06 · Operational workbench

Configurable API connection with session-memory credentials, dark/light appearance, responsive panels, readiness/version visibility and schema mismatch messages.

Primary user journeys

Persona	Workflow and success condition
Document operator	Upload → see parsed/indexed count → ask a supported question → inspect the right source. Errors identify parsing vs. indexing boundaries without deleting the prior document.
AI engineer	Reproduce a failed question → compare retrieval and verification → correlate trace → run labeled evaluation. A latency improvement must preserve expected answer/abstention outcomes.
Reviewer	Open permitted review → inspect reasons and evidence → record one decision. Rejected model claims are not presented as a verified draft.

MVP boundaries

No OCR, conversational memory, unverified response streaming, public sharing, automated answer release, asynchronous job UI or production SSO. The backend has a delete endpoint; a full document library/version-management UI remains future work.

03 / PRODUCT FEATURES

Phase 2: prioritize quality before breadth

Proposed capabilities below are design commitments to evaluate, not features already implemented.

Priority / enhancement	Design direction	Acceptance evidence
P0 · Quality diagnostics	Add per-document parse previews, retrieval labels, calibrated guard evaluation and clear stage-specific failure reasons.	Held-out answerable, absent-fact, conflict and denied-access sets; inspect false review/abstention rates by format.
P0 · Document management	Library, revisions, deletion UX, retention policy and source lineage. Explicit tenant-level admin capability for destructive actions.	Replacement, rollback, authorization and retention tests; deletion invalidates caches and addresses historical review snapshots.
P1 · Durable ingestion jobs	Persist originals in a governed object store; enqueue extraction/index jobs; expose job status and cancellation.	Idempotent retries after worker death; visible terminal state; no partially published version.
P1 · Enterprise identity	OIDC/SSO, short-lived sessions, centralized authorization, audit events and admin separation.	Role/tenant isolation tests across UI, query, upload, review and cache; revocation takes effect across replicas.
P1 · Shared retrieval	Move metadata/reviews to PostgreSQL; evaluate pgvector or a dedicated vector index plus lexical retrieval.	Measured recall/latency under representative load; version-consistent reads and filter enforcement before ranking.
P2 · OCR and rich provenance	Sandbox OCR and preserve page/table/cell coordinates; quality warnings and extraction confidence.	Scanned-PDF and multi-column fixtures; exact page-linked citations and budgeted parsing cost.
P2 · Conversation and safe streaming	Introduce explicit, scoped conversation state. Stream progress first; release answer content only after verification.	No cross-session leakage or unsupported intermediate text; measured time to verified answer.
P2 · Governed tuning UI	Expose only allowlisted top-K/temperature/profile options with bounds and cache-key inclusion.	Server schema validation, policy auditing and repeatable evaluation comparisons for each profile.

Product metrics to instrument

Track answerable-question success, unsupported-answer escape rate, false review rate, citation usefulness, time to verified answer and review resolution time. Segment by document format, corpus version, answer style and tenant class using bounded, privacy-safe labels.

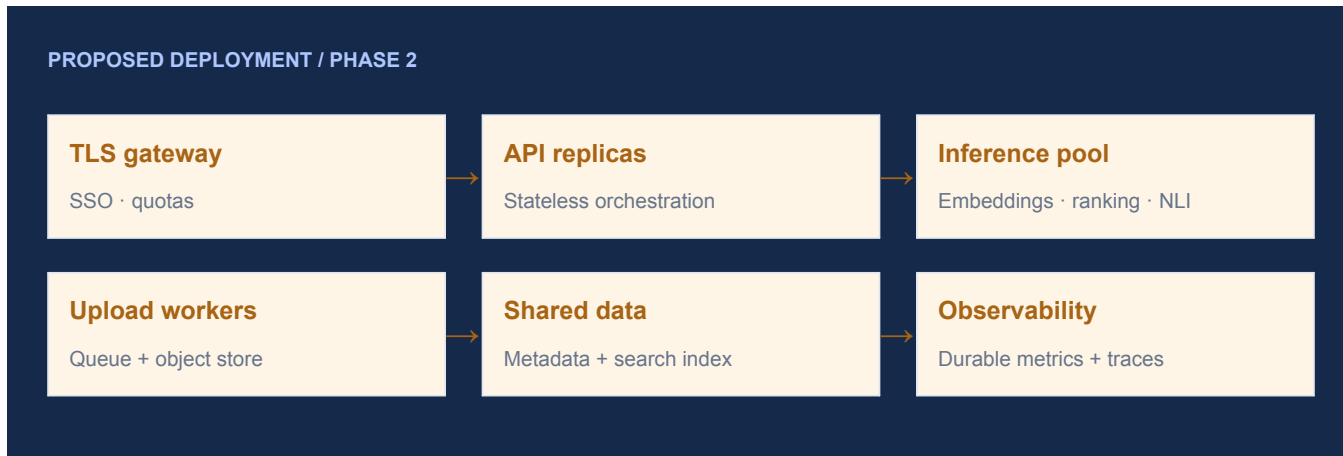
Business measurement

Use a controlled baseline: analyst minutes saved per resolved question × approved labor rate, less inference, infrastructure and review cost. Measure resolution without human help and repeat usage; do not infer business value from low latency or high answer volume alone.

04 / SCALABILITY & SECURITY

Scale the bottleneck, then split the service

Today: one host, bounded corpus, local models. Target: stateless query replicas over versioned shared data.



Migration sequence

- 1. Measure the current envelope.** Load test CPU queueing, model lock contention, full-snapshot memory, exact similarity/BM25 time and SQLite write stalls. Default 20,000 chunks/tenant is a cap, not a demonstrated throughput capacity.
- 2. Externalize shared state first.** Move documents, review records, corpus generations and authorization metadata to a transactional shared database. Replace per-process rate windows with an atomic shared limiter. Do not mount the SQLite file on a shared network filesystem.
- 3. Separate indexing from query serving.** Stage a new document/index generation, compute embeddings asynchronously, then atomically switch the active generation. Use an outbox or durable job record to recover between metadata and search-index commits.
- 4. Scale inference independently.** Batch model requests behind a bounded pool and autoscale on queue delay/utilization. Preload pinned models and warm each worker before readiness. Compare accelerator cost per successful query before choosing hardware.
- 5. Add replicas behind a load balancer.** Use graceful drain and readiness gates. Share or disable response caches until identity, policy and corpus-revision invalidation work consistently across nodes.

Capacity model

Exact retrieval is roughly $O(N \times d)$ for N authorized chunks and embedding dimension d , plus lexical scoring. More API workers also duplicate model memory and local limits; they do not remove those costs.

Availability model

Shared transactional storage needs replication and tested failover. Object-store originals enable re-indexing. Query replicas can recover independently, but provider and model-pool capacity must be included in the SLO.

04 / SCALABILITY & SECURITY

Security follows the evidence boundary

Preserve tenant scope from credentials through retrieval, cache, citations and human review.

Boundary	Implemented control	Production extension / gap
Identity and API	Constant-time bearer comparison; server-owned tenant/groups/roles; query, ingest and review permissions.	Replace static access keys with federated identity and rotation/revocation. Tenant-wide ingest/delete authority needs an explicit admin policy.
Data access	Tenant SQL scope; group intersection before ranking; review visibility requires originating groups to be a subset of reviewer groups.	Enforce matching policy at every shared index/store. Test cache leakage and metadata access; do not rely on UI hiding.
Documents	Size/type/ZIP-expansion checks; defused XML; reduced-environment parser subprocess; no arbitrary URL fetching or Office execution.	Subprocess isolation is not an OS sandbox. Add network-denied containers, CPU/memory quotas and optional malware scanning.
LLM boundary	Strict schemas, evidence-only prompts, citation checks, NLI and safe deterministic rendering. Source text is untrusted data.	Prompt-injection filtering and NLI are probabilistic safeguards, not proof. Add adversarial evaluation and provider data-retention controls.
Secrets and browser	Provider secrets stay server-side. UI token is kept in session memory; text is rendered without raw HTML injection.	Use HTTPS, CSP, a same-origin gateway and managed secrets. In-memory credentials still require XSS protection.
Telemetry and persistence	Bounded route labels and redacted request logs; no routine prompt/source/credential logging. Temp files are removed.	Extracted text, reviews, embeddings and backups remain sensitive. Encrypt disks/backups; define retention, deletion and restricted operational access.

Data lifecycle



Residual data must be managed

Deleting a document removes its indexed chunks; it does not erase historical review payloads, backups or provider-retained data. A production deletion workflow must explicitly cover these stores. Hidden workbook sheets are indexed today, so operators must review their contents before upload.

04 / SCALABILITY & SECURITY

Failure handling without silent degradation

Retry only within time and cost limits. Distinguish unavailable infrastructure from a legitimate evidence gap.

Failure / pressure	Current behavior	Target protocol
Ingress bursts	60 authenticated requests per key per rolling minute, per process; 429 with Retry-After. Default 8 pipeline slots and 2 upload slots; short acquisition timeout yields 503.	Gateway limits before body buffering/auth; shared tenant/key quotas with separate upload/query budgets, fair queues and bounded admission.
Provider 408/429/5xx or transport	Jittered exponential backoff and sequential endpoint fallback; respects remaining deadline/cost reservation and numeric Retry-After. Default 2 attempts per route. Policy refusals do not fail over.	Add circuit breakers and per-route health telemetry. Failover endpoints must satisfy the same data and schema policies.
Invalid model output / evidence	Schema and citation failures cannot release text. Extractive selection rejects unknown IDs. Unsupported/conflicting claims can route to review.	Provider conformance tests and representative guard calibration; preserve fail-closed release semantics.
Index or parser failure	Build before transaction; rollback preserves old chunks. Parsing timeout kills the subprocess and cleans files.	Durable jobs, idempotency keys and staging generations for asynchronous processing. Dead-letter only after bounded retries.
Indexing request timeout	CPU thread work cannot be killed; transaction may finish after browser/API timeout. Stable document ID supports safe replacement on retry.	Expose job status and operation IDs to resolve uncertain outcomes instead of blind resubmission.
Telemetry outage	Bounded export attempts; user requests continue. Development Jaeger traces are memory-backed.	Persistent retention, exporter health alerts and backup/restore tests. Telemetry failure should never authorize unsafe output.

Latency and cost policy

Code defaults: 20 s execution budget, 6 s endpoint timeout, 3 s retrieval and guard budgets, and \$0.10 maximum configured cost per execution. Actual deployment overrides can differ. The cache defaults to 60 s / 128 entries. Provider-reported tokens determine known cost; timed-out or unreported usage increments uncertain attempts rather than pretending cost was zero.

Failover is not “answer at any cost”

Reserve verification time before generation. An exhausted deadline can abstain; degraded ranking requires review. Missing provider usage can be conservatively reserved, but local price settings are estimates rather than a reconciled invoice. Strict wall-clock cancellation is limited by already-running CPU tasks.

04 / SCALABILITY & SECURITY

Release gates and architectural decisions

Promote on measured correctness and recoverability; keep proposed targets separate from observed results.

Quality gate

Version a held-out corpus with supported facts, missing facts, contradictions, numbers, aliases, PDF line wraps and prompt injection. Evaluate retrieval recall separately from model/guard failures; manually inspect false positives and false negatives.

Operational gate

Benchmark cold start and warm query p50/p95/p99 under concurrency; separate cached, uncached and provider-retry runs. Exercise overloaded admission, parser kills, provider outage, review-write failure and telemetry loss.

Decision register

Decision	Reason / consequence
ADR-01 · Modular monolith	One cohesive authorization/verification path; extract storage and inference services when measured pressure justifies it.
ADR-02 · Atomic hybrid store	SQLite keeps lexical and vector records consistent. Migration needs versioned publication; adding replicas alone is unsafe.
ADR-03 · Extractive default	Source-ID selection reduces generated output and paraphrase/repair failures. Fluent synthesis and derived calculations remain a separately evaluated mode.
ADR-04 · Verification before display	No unverified token streaming. Surface stage progress; optimize time to verified answer rather than hiding guard latency.
ADR-05 · Observable outcomes	Business status differs from HTTP success. Monitor answer/review/abstention, billing uncertainty and stage times alongside RED metrics.

Implementation references

References are repository-relative to the inspected project root. These are the authoritative sources for this baseline; the document is not a certification of deployment readiness.

[S1] API & contracts: rag/app.py, rag/config.py, rag/schemas.py.

[S2] Retrieval & storage: rag/ingestion.py, rag/retriever.py.

[S3] Execution & validation: rag/pipeline.py, rag/guardrails.py, rag/evaluator.py.

[S4] File lifecycle: rag/uploads.py, rag/file_parser.py, FILE_UPLOADS.md.

[S5] UI & dependencies: frontend/src/api/client.ts, frontend/src/components/, frontend/package.json, pyproject.toml, uv.lock.

[S6] Operations & evidence: compose.yaml, deploy/otel-collector.yaml, deploy/prometheus.yaml, RUNBOOK.md, VALIDATION.md, selection-regression.json, monitoring-smoke.json.

Next architecture checkpoint

Before multi-tenant production rollout: prove isolation and retention behavior, establish corpus/load targets, provision provider fallback, calibrate quality thresholds and complete a restore drill. Select numeric SLO, RPO and RTO targets with the service owner; none are established by the current small-sample benchmarks.